Vector Search

search by meaning, not by shared words

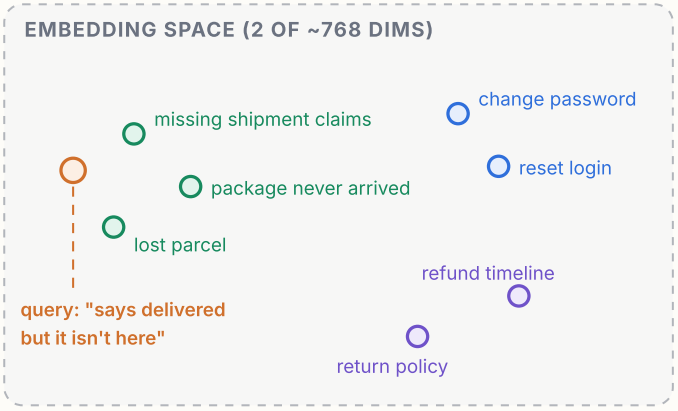
Watch the 4-min video

In one line: an **embedding model** turns text into vectors so that similar meanings land close together; search returns the **nearest neighbours** of the query vector, and at scale uses an **approximate** index such as **HNSW** or **IVF** that trades a little recall for orders of magnitude less work.

1 No shared words, same meaning

A customer types **my package says delivered but it isn't here**. The best help article is titled **Missing shipment claims**. BM25 (lesson 17) sees no common terms and returns nothing useful; a person sees immediately that they mean the same thing. Vector search compares meanings instead of words, which is why it powers semantic search, recommendations, deduplication and the retrieval step of RAG.

2 Embeddings and similarity



Similarity metrics

cosine = a·b / (|a||b|): angle only
dot product = a·b: angle and length
L2 = |a − b|: straight-line distance
unit vectors: cosine = dot,
$L2^2 = 2 - 2 \cdot \text{cosine}$ → same ranking

Rule: use the metric the model was trained with; normalise if it says so.

Same model for documents and queries, or the two spaces don't line up.

Figure 1. Texts with related meaning cluster together, even without shared words. The query lands next to the shipment articles.

An **embedding** is a fixed-length vector of floats produced by a neural model: e.g. 384 dimensions for `all-MiniLM-L6-v2`, 1,536 for OpenAI's `text-embedding-3-small`, 3,072 for `text-embedding-3-large`. The model is trained so that related texts get vectors pointing in similar directions. Key practicalities:

- **Chunking**: models have input limits and one vector per long document blurs its topics, so documents are split into passages (often a few hundred tokens, with some overlap) and each passage is embedded.
- **Same model, same version** for documents and queries. Changing model means **re-embedding everything**: vectors from different models are not comparable. Some models also expect prefixes such as 'query:' and 'passage:'.
- **Metric**: cosine, dot product or L2, as the model card says. For unit-length vectors all three rank identically, so many systems normalise and use dot product, the cheapest.

3 Exact vs approximate nearest neighbours

Exact kNN (brute force) compares the query with every vector and keeps the k closest. It is perfectly accurate and fine for tens of thousands of vectors, or after a filter has already narrowed the set. But 10 million 768-dimension vectors is about 7.7 billion multiply-adds per query, plus reading ~30 GB of floats. **Approximate nearest neighbour (ANN)** indexes visit a tiny fraction of the vectors and return most of the true neighbours. Quality is measured as **recall@k**: the share of the true top k (from exact search) that the ANN result contains. Teams commonly target around 95%, which users rarely distinguish from exact search.

Approach	Work per query	Recall	Memory	Notes
Flat / exact	all N vectors	100%	vectors only	baseline for measuring recall
HNSW	~ a few thousand distance computations, ~log N hops	high (tunable)	vectors + graph links	default in Lucene/Elasticsearch, OpenSearch, pgvector, Qdrant, Weaviate
IVF (IVFFlat)	nprobe lists of ~N/nlist vectors	tunable via nprobe	vectors + centroids	needs training (k-means); often paired with PQ (lesson 23)
DiskANN-style	graph walk, vectors on SSD	high	mostly on disk	for billion-scale on one machine

4 HNSW: a layered graph

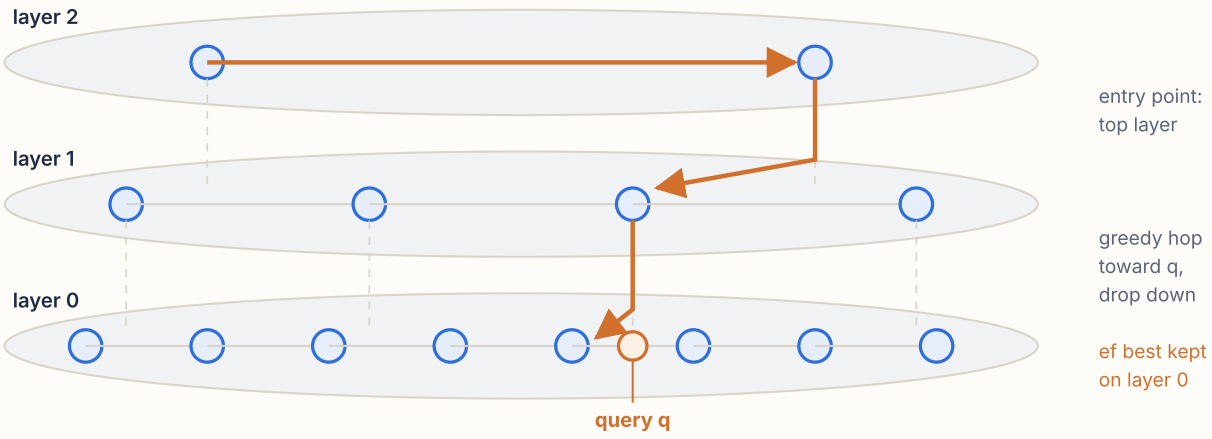
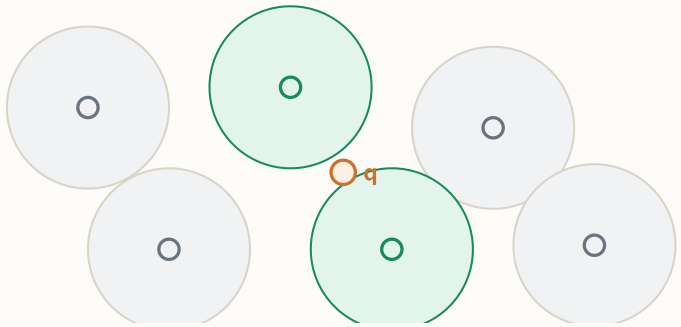


Figure 2. HNSW search: start at the top layer's entry point, hop greedily toward q, drop a layer, repeat; on layer 0 keep the ef best candidates.

Hierarchical Navigable Small World (Malkov and Yashunin, 2016) links each vector to about M of its near neighbours. Each node also gets a random maximum layer, with the probability falling exponentially per layer, so upper layers are sparse 'express lanes' and layer 0 contains everything. A search starts at a fixed entry point on the top layer, repeatedly moves to whichever neighbour is closer to the query, and when no neighbour is closer, drops a layer. On layer 0 it does a best-first search keeping a candidate list of size **ef** and returns the best k. Inserts use the same search to find each new node's neighbours, so HNSW supports incremental adds; deletes are usually tombstones, and heavy churn degrades the graph until it is rebuilt. Its costs: the whole graph and vectors want to live in RAM, and builds are slow. Tuning M, ef_construction and ef_search is lesson 23.

5 IVF: only search nearby clusters



1. offline: k-means → nlist centroids
each vector stored in its nearest list

2. query: find nprobe nearest centroids
(here 2), scan only those lists

3. rule of thumb: nlist ≈ √N to 16√N;
raise nprobe for recall

a true neighbour in an unprobed cell is missed

Figure 3. IVF partitions the space with k-means. Searching 2 of 6 cells does ~1/3 of the work; more probes, more recall.

The **inverted file** index clusters vectors into `nlist` cells at training time. A query computes distances to the centroids, picks the `nprobe` closest cells and scans only their vectors. It is simpler and lighter on memory than HNSW and combines naturally with compression, but needs representative training data, and recall drops for queries near cell borders unless nprobe is raised.

WORKED EXAMPLE: SEMANTIC SEARCH OVER SUPPORT ARTICLES

2,000 help articles are split into ~15,000 passages of a few hundred tokens. Each passage is embedded once with the same model (say 768 dimensions, normalised) and stored with its article ID in an HNSW index; at this size the vectors are only ~46 MB. A customer's question is embedded with the same model (~10–50 ms on a hosted API, less locally), and the index returns the top 5 passages by dot product in a few milliseconds. 'My package says delivered but it isn't here' now retrieves the 'Missing shipment claims' passage without sharing a keyword. The page groups passages by article and shows the top 3 articles. When the team later upgrades the embedding model, they re-embed all passages into a new index and switch over behind an alias (lesson 19).

TRADE-OFFS

- **Meaning vs precision**: great for paraphrases; weak for exact codes, names and rare jargon (lesson 24 combines both).
- **Exact vs ANN**: perfect recall vs orders-of-magnitude speed.
- **HNSW vs IVF**: higher recall at low latency vs less memory and simpler builds.
- **Bigger models** embed better but cost more per query, per vector and in re-embedding time.

COMMON MISTAKES

- Embedding whole long documents as one vector, so specific passages can't be found.
- Mixing vectors from two model versions in one index.
- Using a metric the model wasn't trained for (L2 on unnormalised cosine-trained vectors).
- Never measuring recall against exact search, so a misconfigured index goes unnoticed.

6 Key terms

Embedding: vector representation of meaning from a model

kNN / ANN: exact / approximate k nearest neighbours

HNSW: multi-layer proximity graph, greedy search

Cosine similarity: angle between vectors, ignores length

Recall@k: share of the true top k that ANN returns

IVF: k-means cells; search only nprobe nearest cells

7 Interview questions

Q1. How does HNSW work at a high level? BASIC

It builds a layered graph of vectors linked to their neighbours; a search starts at the sparse top layer, moves greedily toward the query and drops down to denser layers, keeping ef candidates at the bottom.

Q2. Exact vs approximate search? BASIC

Exact compares every vector and is perfectly accurate but slow at scale; approximate indexes visit a small fraction and trade a little recall for huge speed.

Q3. Cosine, dot product or L2? BASIC

Whichever the embedding model was trained for; on normalised vectors they produce the same ranking, so dot product is typical.

Q4. You switch to a better embedding model. What has to happen? SCENARIO

Re-embed every document with the new model into a new index, because old and new vectors live in different spaces, then cut over (e.g. via an alias) and re-tune.

Q5. Only 20,000 vectors. Do you need an ANN index? SCENARIO

Probably not: brute force over 20,000 vectors takes milliseconds and gives exact results; ANN pays off at hundreds of thousands to millions.

Q6. Recall is 95%. Does that mean 5% of answers are wrong? TRICKY

No: 5% of the true top-k neighbours are replaced by slightly farther ones, which are usually still relevant; measure end-task quality too.

Q7. Why does vector search fail on 'XB-2017'? TRICKY

Embeddings capture meaning, and near-identical codes like XB-2017 and XB-2019 map to nearly the same vector; exact tokens need keyword search or hybrid (lesson 24).

Next: SD-P-010 · Vector Index Tuning backendroadmap.com